Java Core — Bản đồ tổng quan: Collections, Stream, Collectors, Optional

> Tài liệu tra cứu nhanh cho lộ trình 33 bài. Các mốc `@since` trong file được kiểm từ source JDK 25 (`$JAVA_HOME/lib/src.zip`), không ghi theo trí nhớ. Project đang dùng Java 25 nên mọi thứ trong đây đều dùng được.

---

0. Ba cái tên dễ nhầm — đọc trước tiên

Tên	Loại	Nằm ở	Dùng để
`Collection`	**interface** gốc	`java.util`	Kiểu khai báo: `void xuLy(Collection<User> ds)`
`Collections`	**class tiện ích** (static)	`java.util`	`Collections.sort()`, `unmodifiableList()`, `emptyList()`
`Collectors`	**class tiện ích** (static)	`java.util.stream`	`toMap()`, `groupingBy()` — dùng với `stream().collect()`

`Collections` và `Collectors` **không liên quan gì nhau**, chỉ trùng tên.

---

1. Bản đồ tổng thể — 3 tầng



Quy tắc vàng: `Collection` là **danh từ** (cái kho). `Stream` là **động từ** (đường ống). Stream không lưu trữ gì — nó chỉ chảy qua.

---

2. Collections Framework

2.1 Cây kế thừa

```
Iterable
└── Collection
    ├── List          thứ tự + trùng lặp + có index
    ├── Set           không trùng lặp
    └── Queue         hàng đợi
        └── Deque     hai đầu

Map                   ★ KHÔNG kế thừa Collection
```

> `Map` **không phải** `Collection`: không có `add()`, không `iterator()` trực tiếp. Muốn duyệt phải qua `entrySet()` / `keySet()` / `values()`. Lý do: `Collection` là "tập phần tử", `Map` là "tập **cặp**".

2.2 List

Implementation	get(i)	add cuối	add/remove giữa	contains	Khi nào dùng
`ArrayList`	O(1)	O(1)*	O(n)	O(n)	**Mặc định — 95% trường hợp**
`LinkedList`	O(n)	O(1)	O(1) nếu đã có iterator	O(n)	Gần như không bao giờ; cần Deque thì dùng `ArrayDeque`
`List.of(...)`	O(1)	✕ bất biến	✕	O(n)	Hằng số, dữ liệu không đổi

* amortized (thỉnh thoảng phải cấp lại mảng)

2.3 Set

Implementation	Thêm/Tìm	Thứ tự	Khi nào dùng
`HashSet`	O(1) tb	**không hứa**	Mặc định — chỉ cần khử trùng
`LinkedHashSet`	O(1) tb	**thứ tự chèn**	Khử trùng nhưng vẫn giữ thứ tự gốc
`TreeSet`	O(log n)	**sắp theo giá trị**	Cần sắp xếp / `first()` / `headSet()` / `ceiling()`
`EnumSet`	rất nhanh (bitmask)	thứ tự enum	Key là `enum` — luôn ưu tiên

2.4 Queue / Deque

Implementation	Vai trò	Lấy ra phần tử nào	Ghi chú
`ArrayDeque`	Stack **và** Queue	đầu hoặc cuối, O(1)	**Thay cho** `Stack` **cũ** (Stack là legacy, synchronized vô ích)
`LinkedList`	Queue	đầu	Dùng được nhưng `ArrayDeque` nhanh hơn
`PriorityQueue`	hàng đợi ưu tiên	**nhỏ nhất** theo Comparator	`offer` / `poll` O(log n); **duyệt KHÔNG theo thứ tự**

> `PriorityQueue.toString()` / `forEach` in ra **lộn xộn** — chỉ `poll()` mới đúng thứ tự ưu tiên.

2.5 Map

Implementation	get/put	Thứ tự key	Khi nào dùng
`HashMap`	O(1) tb	**không hứa**	Mặc định
`LinkedHashMap`	O(1) tb	**thứ tự chèn**	Giữ thứ tự để hiển thị / làm LRU cache
`TreeMap`	O(log n)	**sắp theo key**	Cần sắp xếp, hoặc `firstKey()` / `floorKey()` / `subMap()`
`EnumMap`	rất nhanh (mảng)	thứ tự enum	Key là `enum` — luôn ưu tiên
`ConcurrentHashMap`	O(1) tb	không hứa	Nhiều thread ghi

2.6 Method của Map hay dùng (Java 8+)

Method	Ý nghĩa
`getOrDefault(k, def)`	Không có key thì trả `def`, không trả null
`putIfAbsent(k, v)`	Chỉ đặt khi chưa có
`computeIfAbsent(k, f)`	Chưa có thì **tính rồi đặt** — chuẩn cho `Map<K, List<V>>`
`computeIfPresent(k, f)`	Có rồi mới tính lại
`compute(k, f)`	Tính trong mọi trường hợp
`merge(k, v, f)`	Chưa có thì đặt `v`, có rồi thì gộp bằng `f` — chuẩn cho đếm tần suất
`forEach((k,v) -> ...)`	Duyệt gọn
`entrySet()`	**Cách duyệt rẻ nhất** khi cần cả key lẫn value

> Đừng duyệt `keySet()` rồi `get(k)` trong vòng lặp — tra bảng băm hai lần. Dùng `entrySet()`.

---

3. Stream API

3.1 Tạo Stream

Cách	Ví dụ
Từ Collection	`list.stream()`, `set.stream()`
Từ Map	`map.entrySet().stream()`, `map.values().stream()`
Từ mảng	`Arrays.stream(arr)`
Từ giá trị rời	`Stream.of("a", "b")`
Từ 1 giá trị có thể null	`Stream.ofNullable(x)` *(9)*
Vô hạn	`Stream.iterate(1, n -> n+1)`, `Stream.generate(Math::random)`
Vô hạn có điều kiện dừng	`Stream.iterate(1, n -> n<100, n -> n*2)` *(9)*
Số nguyên thuỷ	`IntStream.range(0, 10)`, `IntStream.rangeClosed(1, 10)`
Từ Optional	`optional.stream()` *(9)*
Nối 2 stream	`Stream.concat(a, b)`

3.2 Intermediate operations (lazy — trả về `Stream`)

Method	Tác dụng	@since
`filter(Predicate)`	Giữ phần tử thoả điều kiện	8
`map(Function)`	Biến đổi 1 → 1	8
`flatMap(Function)`	Biến đổi 1 → n rồi **làm phẳng**	8
`mapMulti(BiConsumer)`	Như flatMap nhưng không tạo Stream con	**16**
`distinct()`	Khử trùng (dùng `equals` / `hashCode`)	8
`sorted()` / `sorted(Comparator)`	Sắp xếp — **stateful, phải gom hết**	8
`limit(n)` / `skip(n)`	Cắt đầu / bỏ đầu	8
`takeWhile(p)` / `dropWhile(p)`	Lấy/bỏ **cho tới khi** điều kiện sai	**9**
`peek(Consumer)`	Nhìn trộm — **chỉ để debug**	8
`mapToInt/Long/Double`	→ stream nguyên thuỷ (tránh boxing)	8
`boxed()`	Stream nguyên thuỷ → `Stream<Integer>`	8
`gather(Gatherer)`	Toán tử trung gian **tự viết**	**24**
`parallel()` / `sequential()`	Đổi chế độ chạy	8

3.3 Terminal operations (chạy thật — kết thúc stream)

Nhóm	Method	Trả về
Gom	`collect(Collector)`	tuỳ Collector
	`toList()` *(16)*	`List` **bất biến** (`unmodifiableList`, cho phép null)
	`toArray()`	mảng
Tìm	`findFirst()` / `findAny()`	`Optional<T>`
	`min(cmp)` / `max(cmp)`	`Optional<T>`
Kiểm tra	`anyMatch` / `allMatch` / `noneMatch`	`boolean` (short-circuit)
Đếm/Tính	`count()`	`long`
	`reduce(...)`	`Optional<T>` hoặc `T`
	`sum()` / `average()` / `summaryStatistics()`	chỉ có ở stream nguyên thuỷ
Lặp	`forEach` / `forEachOrdered`	`void`

3.4 Ba tính chất phải nhớ

1. **Lazy** — không có terminal op thì **không chạy gì cả**.
2. **Chảy theo chiều dọc** — mỗi phần tử đi hết pipeline rồi mới tới phần tử sau

(không phải `map` xong hết rồi mới `filter`).

1. **Dùng một lần** — gọi lại stream cũ: `IllegalStateException: stream has already

been operated upon or closed`.

3.5 Stream ≠ RxJS

	RxJS `Observable`	Java `Stream`
Mô hình	**push** (producer đẩy)	**pull** (terminal op kéo)
Đồng bộ	bất đồng bộ	**đồng bộ, blocking**
Thời gian	`debounce`, `delay`, `interval`	**không có**
Kênh lỗi	`onError`, `retry`, `catchError`	ném exception thẳng
Huỷ	`unsubscribe()`	**không huỷ được**

Bản đồ ghép đầy đủ:

	Đồng bộ	Bất đồng bộ (Project Reactor / Spring WebFlux)
0..1	`Optional<T>`	`Mono<T>`
0..n	`Stream<T>`	`Flux<T>`

---

4. Collectors

4.1 Gom vào Collection

Collector	Kết quả	Ghi chú
`toList()`	`List`	Javadoc: **không cam kết** type / mutability / serializability. Thực tế là `ArrayList` nhưng đừng dựa vào
`toUnmodifiableList()` *(10)*	`List` bất biến	
`toSet()` / `toUnmodifiableSet()` *(10)*	`Set`	
`toCollection(TreeSet::new)`	kiểu bạn chọn	Khi cần đúng implementation

4.2 `toMap` — 3 overload

```
toMap(key, value)                    // key TRÙNG -> IllegalStateException
toMap(key, value, merge)             // merge quyết định giữ cái nào
toMap(key, value, merge, mapFactory) // + chọn loại Map (BẮT BUỘC có merge)
```

Bẫy	Hậu quả
Key trùng (bản 2 tham số)	`IllegalStateException`
`valueMapper` trả `null`	`NullPointerException`
`mapFactory` = `TreeMap::new`	Sắp theo **key**, **vứt bỏ** thứ tự `sorted()` của stream
Cần giữ thứ tự stream	Dùng `LinkedHashMap::new`

> Key **không** duy nhất về bản chất (phòng ban, trạng thái) → **dùng** `groupingBy`, đừng vá bằng `(a, b) -> a`.

4.3 Nhóm và chia

Collector	Kết quả	Ghi chú
`groupingBy(f)`	`Map<K, List<T>>`	n phần tử → 1 key
`groupingBy(f, downstream)`	`Map<K, R>`	Lồng collector khác vào
`groupingBy(f, mapFactory, downstream)`	`Map<K, R>`	Chọn `EnumMap` / `TreeMap`
`partitioningBy(p)`	`Map<Boolean, List<T>>`	**Luôn có đủ 2 key** `true` / `false`
`partitioningBy(p, downstream)`	`Map<Boolean, R>`	

> Khác biệt đáng giá: `partitioningBy` luôn có cả `true` và `false` (list rỗng nếu không ai thoả), nên `.get(true).size()` an toàn. `groupingBy` thì key không có là `null`.

4.4 Downstream collectors (dùng bên trong groupingBy/partitioningBy)

Collector	Kết quả	@since
`counting()`	`Long`	8
`summingInt/Long/Double(f)`	tổng	8
`averagingInt/Long/Double(f)`	`Double`	8
`summarizingInt/Long/Double(f)`	count, sum, min, max, average **một lượt**	8
`minBy(cmp)` / `maxBy(cmp)`	`Optional<T>`	8
`mapping(f, downstream)`	biến đổi trước khi gom	8
`filtering(p, downstream)`	lọc **giữ key rỗng** (khác filter trước groupingBy)	**9**
`flatMapping(f, downstream)`	làm phẳng rồi gom	**9**
`reducing(...)`	rút gọn tuỳ ý	8
`collectingAndThen(c, finisher)`	gom xong rồi biến đổi lần cuối	8
`teeing(c1, c2, merger)`	chạy **2 collector cùng lúc** rồi gộp	**12**

4.5 Chuỗi

Collector	Kết quả ví dụ
`joining()`	`"AnBaoCuc"`
`joining(", ")`	`"An, Bao, Cuc"`
`joining(", ", "[", "]")`	`"[An, Bao, Cuc]"` — stream rỗng vẫn ra `"[]"`

---

5. Optional

5.1 Bản chất

```
private final T value;                              // null = rỗng
public boolean isPresent() { return value != null; }
```

Một cái hộp chứa **0 hoặc 1** giá trị. Tổng cộng 20 method. **Không** phải `Collection`, **không** `Iterable`.

5.2 Ba nhóm API

A. Đưa vào hộp

Method	Khi nào
`Optional.of(x)`	Chắc chắn không null — null thì **nổ NPE ngay**
`Optional.ofNullable(x)`	Có thể null — null thành `empty()`
`Optional.empty()`	Rỗng tường minh

B. Biến đổi trong hộp (`Optional` → `Optional`)

Method	Khi nào	@since
`map(f)`	`f` trả về **giá trị** — `f` trả null thì ra `empty()`	8
`flatMap(f)`	`f` trả về **Optional** (nếu dùng map sẽ ra `Optional<Optional<T>>`)	8
`filter(p)`	Không thoả → thành `empty()`	8
`or(supplier)`	Rỗng thì thử nguồn khác, **vẫn trả Optional**	**9**

C. Lấy ra khỏi hộp

Method	Trả về	Lưu ý
`orElse(giaTri)`	`T`	⚠️ Vế phải **luôn được tính**, kể cả khi có giá trị
`orElseGet(supplier)`	`T`	Chỉ chạy khi rỗng
`orElseThrow()` *(10)*	`T`	`NoSuchElementException`
`orElseThrow(supplier)`	`T`	**Cách chuẩn ở service layer**
`ifPresent(consumer)`	`void`	Side-effect
`ifPresentOrElse(c, r)` *(9)*	`void`	Xử lý cả 2 nhánh
`stream()` *(9)*	`Stream<T>`	Cầu nối sang Stream API
`get()`	`T`	❌ JDK khuyên dùng `orElseThrow()` thay thế

5.3 `orElse` vs `orElseGet` — nhìn chữ ký là hiểu

```
public T orElse(T other)                    // other là GIÁ TRỊ -> phải tính TRƯỚC
public T orElseGet(Supplier<? extends T> s)  // s là HÀM      -> chỉ gọi khi rỗng
```

> **Quy tắc**: vế phải là hằng/biến sẵn có → `orElse` . Là **lời gọi hàm** → `orElseGet` . Không chỉ là hiệu năng: nếu hàm đó query DB / gọi API / ghi log, `orElse` gây **tác dụng phụ** ngay cả khi Optional có giá trị.

5.4 Dùng ở đâu

Javadoc JDK nói rõ: "*primarily intended for use as a **method return type**... A variable whose type is Optional should **never itself be null**.*"

Vị trí	Dùng?	Thay bằng
Kiểu **trả về** của method tra cứu	✅	—
Field của class / JPA entity	❌	Kiểu thật, cho phép null
Tham số method	❌	Nạp chồng hàm, hoặc nhận null
Phần tử trong `List` / `Map`	❌	Bỏ hẳn phần tử đó
`Optional<List<T>>`	❌	Trả `List` rỗng — rỗng đã là "không có"
Vòng lặp nóng (triệu lần/giây)	⚠️	Mỗi lần là 1 object mới

Vì sao không làm field:

1. Field `Optional` **vẫn có thể null** → hai cách biểu diễn "không có", tệ hơn lúc đầu.
2. `Optional` **không** implement `Serializable`.
3. JPA / Jackson không map field `Optional` ra cột / ra JSON một cách tự nhiên.

5.5 Bốn cách dùng sai hay gặp khi review

Sai	Đúng
`opt.get()` không kiểm tra	`opt.orElseThrow(() -> new XxxNotFoundException(id))`
`if (opt.isPresent()) { opt.get()... } else {...}`	`opt.map(...).orElse(...)`
`opt.orElse(taoObjectTonKem())`	`opt.orElseGet(this::taoObjectTonKem)`
`.filter(Optional::isPresent).map(Optional::get)`	`.flatMap(Optional::stream)`

Ngoại lệ hợp lệ: dùng `isPresent()` / `isEmpty()` **một mình** để rẽ luồng thì không sao — `if (repo.findByEmail(e).isPresent()) throw new EmailDaTonTaiException(e);`

---

6. Bảng chọn nhanh

Tình huống	Dùng
Danh sách có thứ tự, truy cập theo index	`ArrayList`
Khử trùng lặp	`HashSet`
Khử trùng nhưng giữ thứ tự gốc	`LinkedHashSet`
Tra cứu theo khoá	`HashMap`
Khoá là `enum`	`EnumMap` / `EnumSet`
Cần key sắp xếp, tra khoảng	`TreeMap` / `TreeSet`
Stack hoặc Queue	`ArrayDeque`
Hàng đợi theo độ ưu tiên	`PriorityQueue`
Biến đổi/lọc tập dữ liệu	`stream()`
Kết quả 0..n	trả `List` (rỗng nếu không có)
Kết quả 0..1, "không có" cần được chú ý	trả `Optional`
Key duy nhất (id, email)	`Collectors.toMap`
Key không duy nhất (phòng ban, trạng thái)	`Collectors.groupingBy`
Chia đôi theo điều kiện boolean	`Collectors.partitioningBy`
Cần count + sum + min + max + avg	`Collectors.summarizingXxx`

7. Đối chiếu với TypeScript / JavaScript

Java	TS/JS
`List<T>`	`T[]`
`Map<K,V>`	`Map<K,V>` / object
`Set<T>`	`Set<T>`
`stream().filter().map()`	`arr.filter().map()` (nhưng JS **không lazy**)
`collect(toList())`	`[...]`
`collect(toMap(k, v))`	`Object.fromEntries(arr.map(x => [k, v]))`
`collect(groupingBy(f))`	`Object.groupBy(arr, f)`
`collect(joining(", "))`	`arr.join(", ")`
`reduce(...)`	`arr.reduce(...)`
`Optional<T>`	`T | null` (nhưng là **object runtime**, không phải kiểu compile-time)
`Mono<T>` / `Flux<T>`	`Promise<T>` / `Observable<T>`

Khác biệt lớn nhất: **Java không có** `strictNullChecks`. `Optional` mô phỏng điều đó bằng *kiểu dữ liệu*, nên nó chỉ **nhắc**, không **ép** — bạn vẫn `.get()` bừa được.

8. Bẫy tổng hợp

Bẫy	Triệu chứng
Sửa Collection khi đang duyệt	`ConcurrentModificationException`
`List.of(...)` rồi gọi `add()`	`UnsupportedOperationException`
`stream.toList()` rồi gọi `add()`	`UnsupportedOperationException` (khác `collect(toList())`)
Dùng lại Stream đã chạy	`IllegalStateException`
`toMap` key trùng	`IllegalStateException`
`toMap` value null	`NullPointerException`
`Optional.of(null)`	`NullPointerException` ngay dòng tạo
`optional.get()` khi rỗng	`NoSuchElementException`
`orElse(hamTonKem())`	Hàm vẫn chạy dù Optional có giá trị
Duyệt `PriorityQueue` bằng `forEach`	Thứ tự **không** đúng ưu tiên
`double` cho tiền	Sai số — dùng `BigDecimal` hoặc `long` (đơn vị nhỏ nhất)
`parallelStream()` cho tác vụ I/O	Nghẽn common ForkJoinPool toàn JVM

9. Code mẫu trong repo này

Chủ đề	File
List / Set	`javacore33/collections/list_set/`
Map, đếm tần suất	`javacore33/collections/map/`
Queue / Deque	`javacore33/collections/queue/`, `collections/deque/`
Stream với số	`javacore33/streams/so/StreamSoDemo.java`
Stream với object, `groupingBy`	`javacore33/streams/nhanvien/StreamObjectDemo.java`
Collectors nâng cao, `toMap`	`javacore33/streams/nhanvien/CollectorsNangCaoDemo.java`
Method reference	`javacore33/streams/nhanvien/MethodReferenceDemo.java`
Optional	`javacore33/streams/optional/OptionalDemo.java`
Chọn implementation	`api/collections_and_streams/collections_framework/t12_ChoosingImplementations.java`